

A 3.8 μ W 10-Keyword Noise-Robust Keyword Spotting Processor Using Symmetric Compressed Ternary-Weight Neural Networks

Bo Liu, *Member, IEEE*, Na Xie, Renyuan Zhang, Haichuan Yang,
Ziyu Wang, Deliang Fan, *Member, IEEE*, Zhen Wang,
Weiqiang Liu, *Senior Member, IEEE*, Hao Cai, *Senior Member, IEEE*,

Abstract—A ternary-weight neural network (TWN) inspired keyword spotting (KWS) processor is proposed to support complicated and variable application scenarios. To achieve high-precision recognition of 10 keywords under 5dB~Clean wide range of background noises, a convolution neural network consists of 4 convolution layers and 4 fully connected layers, with modified sparsity-controllable Truncated Gaussian Approximation based ternary-weight training is used. End to end optimization composed of three techniques are utilized: 1) the stage-by-stage bit-width selection algorithm to optimize the hardware overhead of FFT; 2) the lossy compressed TWN with symmetric kernel training (SKT) and dedicated internal data reuse computation flow; 3) the error inter-compensation approximate addition tree to reduce the computation overhead with marginal accuracy loss. Fabricated in an industrial 22-nm CMOS process, the processor realizes up to 10 keywords real-time recognition under 11 background noise types, with the accuracy of 90.6% @clean and 85.4% @5dB. It consumes an average power of 3.8 μ W at 250KHz and the normalized energy efficiency is 2.79 $\times$ higher than state-of-the-art.

Index Terms—Ternary-weight neural network, speech recognition, approximate computing, error inter-compensation.

I. INTRODUCTION

THE keyword spotting (KWS) processor is a widely used speech-triggered interface for human-machine interaction, which plays an important role in wearable and mobile devices. Multi-keyword and various background noises supporting can face a wide range of real-world scenarios, thus takes intensive research [1].

Deep neural network (DNN) has demonstrated prominent advantage in speech recognition [2]. It mainly consists of feature extraction (FE) and DNN modules for words classification [3]. State-of-the-art architectures present design challenges in terms of ultra-low power consumption and real-time processing due to the computationally intensive requirements for achieving high recognition accuracy and noise robustness [4]–[11]. In K.Kim's work [11], the gated recurrent unit

(GRU) based recurrent neural network (RNN) digital classifier consumes the power of 13.6 μ W. A DNN with 1 long and short term memory (LSTM) layer and 1 fully connected (FC) layer is proposed in [1]. The power of the classifier is 5 μ W, but it can only applied in the near microphone scene. Quantization is an efficient way to realize the balance between recognition accuracy and power consumption [12], [13]. Based on the binarized depthwise separable convolution neural network (DSCNN), the power consumption of the chip can be reduced to 0.51 μ W, however, it can only support 1~2 keywords recognition [14].

Another efficient way to reduce the power consumption is to optimize the computing circuits. Since DNNs are inherently error-tolerant, approximate computing becomes a proper technique to reduce power consumption with acceptable accuracy loss [15]–[17]. Approximate design technologies and hardware architectures have been proposed in [18]–[22], e.g. approximate adder/multiplier, can effectively reduce the computing overhead.

In this work, we propose a ternary weight network (TWN)-based noise-robust KWS processor, which is fabricated with an industry 22-nm technology. To our best knowledge, this is the first fabricated KWS processor that can realize 10 keywords (12 classes) high-accuracy real-time keywords recognition with the power consumption of 3.8 μ W. It can support 11 background noise types with 5dB~Clean wide range of signal-noise ratios (SNR). Our main contributions include:

- 1) A stage-by-stage bit-width selection algorithm (SBSA) for serial pipeline fast fourier transform (FFT) to reduce the power consumption of FE. Bit-width of each stage is precisely selected by considering signal to quantization noise ratio (SQNR).
- 2) A compressed TWN with lossy configurable Symmetric Kernel Training (SKT), which can support high recognition accuracy under a wide range of background noises without the cost of large memory capacity.
- 3) An error inter-compensation approximate addition tree with innovative Cartesian Genetic programming (CGP) based error inter-compensation scheme to further reduce the power consumption of TWN accelerator.

The remainder of this article is organized as follows. Section II introduce the background and challenges of TWN-based KWS system. Three proposed techniques are described

This work is supported by National Natural Science Foundation of China under Grant 62274029 and U19A2074. (Bo Liu and Hao Cai contributed equally to this work).

B. Liu, N. Xie, R. Zhang, H. Yang, Z.Y. Wang and H. Cai are with National ASIC System Engineering Center, Southeast University, Nanjing, 210096, China. D. Fan is with Johns Hopkins University, Baltimore, MD, USA. Z. Wang is with Nanjing Prochip Electronic Technology Co. Ltd, Nanjing, China. W. Liu is with College of Electronic and Information Engineering, Nanjing University of Aeronautics and Astronautics, Nanjing 210016, China.

Corresponding author: Hao Cai (Email: hao.cai@seu.edu.cn).

in Sections III-V. Section VI presents the chip implementation and measurement results. Section VII concludes this article.

II. CHALLENGES AND OUR SOLUTIONS

A typical digital DNN-based KWS system is shown in Fig. 1 (a). After the speech signal received by microphone, it goes through the Mel Frequency Cepstral Coefficient (MFCC) for feature extraction, then performs the neural network calculation, and finally get the classification result. In this process, MFCC, data/weight storage and NN computation are the three main stages with the largest energy consumption in hardware processing. In the TWN-based KWS system, each stage poses different challenges.

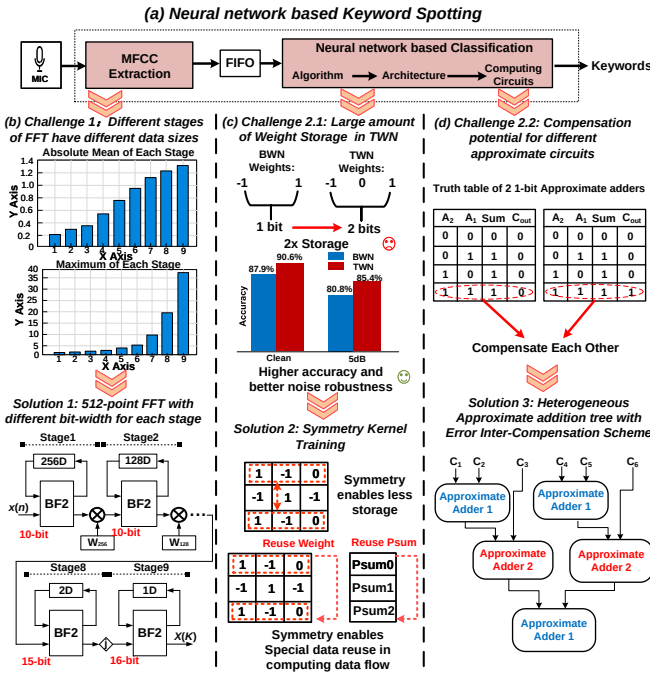


Fig. 1. End-to-end design challenges and our solutions in TWN-based KWS system.

A. FFT optimization for noise-robust low power KWS

FFT is one of the key processing units of MFCC for speech feature extraction, which consumes most of the power due to the huge amount of computation and storage. The power consumption of FFT accounts for 62.1% in MFCC, while in the KWS system, the power consumption of MFCC accounts for 33.8% [23]. Therefore, optimizing FFT calculation is an efficient way to reduce the power consumption. Nowadays, there are mainly three different structures to implement FFT: parallel, serial pipeline and memory-based loop recursion. Among them, serial pipeline FFT can achieve a better balance between hardware resources and throughput, which is more suitable for our lightweight KWS system. The core idea of serial pipeline FFT is multi-stage processing. For the complete N -point radix-2 FFT, it is decomposed into $\log_2 N$ level butterfly operations. The butterfly operation of each stage does not interfere with each other, thus different stages can be calculated with different data bit-width. Fig. 1(b) shows the

distribution of the maximum value and mean value of the each stage results of FFT in the speech “left”. It can be seen that the data sizes of different stages in FFT is different, how to precisely determine the data bit-width in each stage to achieve the balance between accuracy and hardware resource becomes a challenge.

B. TWN hardware implementation and optimization with approximate computing

Quantization is an efficient way for DNN hardware implementation. Recent works quantized the activations/weights to binary, thus eliminate the multiplication and alleviate the data storage [14], [23], [24]. However, binarized data leads to severe loss of information. Compared with the binary weight network (BWN) in [23], TWN has a stronger weight expression ability [25]. It can achieve higher recognition accuracy under more complex classification tasks, shown in Fig. 1(c).

$$W_i^t = f_t(W_i|\delta) = \begin{cases} +a, & \text{if } W_i > \delta \\ 0, & \text{if } |W_i| \leq \delta \\ -a, & \text{if } W_i < -\delta \end{cases} \quad (1)$$

As shown in (1), where δ is the threshold and a is the scaling factor. The ternary-weight quantization scheme usually uses a scaling factor a multiplied by $+1$, -1 , 0 to represent the weight, it has an additional 0 value compared to BWN. However, 0 does not affect the multiplication and addition operations, it increases the model sparsity and enables even less computation. Since each weight in TWN is 2-bit, this brings $2\times$ storage overhead compared to BWN, how to further compress the model becomes a challenge. Joint optimization of algorithm and hardware in TWN can realize the KWS system with lower power consumption, on the basis of obtaining higher accuracy and wider noise adaptation range than BWN.

Besides, there are a large amount of addition operations in TWN, designing advanced approximate adders is an effective way to reduce computing power consumption. How to minimize hardware resource while maintaining the accuracy is another challenge. Typical approximate adders are of a single design, however, there is a potential for mutual compensation between different approximate components. Fig. 1(d) shows two 1-bit approximate half adders as an example. When both inputs are 1, the original wrong calculation results can be repaired to exact results through mutual compensation. Compensating with different approximate adders can even achieve higher accuracy while consuming less power. However, the compensation of calculation results is a probabilistic event, the complete compensation is difficult to achieve. Therefore, an effective compensation scheme is required to design the optimal compensation circuit.

C. Our end-to-end Solutions

The bottom of Fig. 1 shows our end-to-end solutions for these challenges, from feature extraction to neural network computation. First, we propose the 512-point serial FFT with different bit-width in each stage, to reduce the hardware overhead of MFCC. Second, in neural network based classification, we propose a novel symmetry kernel training method

for TWN to reduce the weight storage and computations. Symmetrical weights enable more efficient model compression at the algorithm level, and unique data reuse and data flow at the architecture level. What's more, at the circuit level, a heterogeneous approximate adder tree is proposed to reduce the power consumption of DNN computation with tiny accuracy loss. A general error inter-compensation scheme is proposed to solve the problem of statistically optimal error compensation between different approximation components.

III. STAGE-BY-STAGE BIT-WIDTH SELECTION ALGORITHM FOR SERIAL PIPELINE FFT

Since the FFT unit requires a large amount of intermediate data buffers for data regularization or temporary storage, reducing the data bit-width can significantly reduce the FFT calculation overhead. We propose the SBSA to precisely determine the data bit-width of each stage. In this section, we introduce the details of SBSA with an example. Based on the optimized FFT module, the frame-length adaptive based MFCC is given, and we show how it works in the TWN based KWS system.

First, 512-point radix-2 single-path delay feedback fast fourier transform (R2SDF-FFT) proposed in [26] is utilized in this work. In the serial pipeline FFT structure, the previous stages occupy a larger amount of hardware resource, and are more error tolerant, thus it has a larger optimization space and optimization benefits than the post stages. The signal to quantization noise ratio (SQNR) is used to measure the overall error introduced by the bit-width truncation, the larger SQNR indicates the more accurate computation result. The SQNR is defined in (2), where $X(k)$ represents the exact N-point FFT calculation result of double floating-point number, while $\bar{X}(k)$ represents the approximate calculation result after bit-width truncated. MSE is the mean square error.

$$MSE = \frac{1}{N} \sum_{k=0}^{N-1} (X(k) - \bar{X}(k))^2 \quad (2)$$

$$SQNR = 10 \lg \frac{\max(X(k))^2}{MSE}$$

The pseudocode of SBSA is shown in Algorithm 1. According to our previous work [27], the initial bit-width of each stage $Wd_initial$ is set as 16. The data format is fixed-point and the number of fractionary bits is 9 in the FFT calculation. The $SQNR_bound$ represents the tolerable error boundary. When reducing the bit-width of the current stage, the bit-width of other stages are unchanged, and the bit-width of each stage is optimized from the first stage within the error boundary. It should be noted that the lowest bit-width Wd_limit of each stage is theoretically the bit-width of the input data, i.e., the output bit-width of the previous stage. The Wd_limit of the first stage is the bit-width of the original speech data. For example, for one speech signal of GSCD, we gradually reduce the bit-width of the first stage and calculate the SQNR until the set error boundary is exceeded. After the bit width of the first stage is determined, the minimum bit-width of the second stage is set as the bit-width of the first stage, and continue to

optimize the bit-width of the second stage. The rest of the stages are performed the same. In this way, we achieve trade-off between hardware optimization and accuracy evaluation.

Algorithm 1: Stage-by-stage bit-width selection algorithm for serial N-point pipeline FFT.

Input: Initial data bit-width: $Wd_initial$;
Tolerable error boundary: $SQNR_bound$;
Output: The bit-width of the k_{th} stage: $Wd_stage(k)$;

```

1  $Wd\_stage \leftarrow Wd\_initial$ 
2  $Wd\_limit(1) \leftarrow$  bit-width of the original speech data
  //set the first stage minimum bit-width;
3  $k \leftarrow 1$  // optimize from the first stage
4 while  $k \leq \log_2 N$  do
5   Calculate  $SQNR$  from one of the GSCD dataset
6   if ( $SQNR < SQNR\_bound$  &&
        $Wd\_stage(k) > Wd\_limit(k)$ ) then
7      $Wd\_stage(k) - -$ ;
8   else
9      $Wd\_limit(k+1) = Wd\_stage(k)$ ;
10    //set the next stage minimum bit-width;
11     $k + +$ ;
12  end
13 end
14 Return:  $Wd\_stage$ 
```

As the error boundary decreases, the hardware overhead of FFT will decrease, but the accuracy will also deteriorate, so the error boundary needs to be set reasonably. After applying the stage-by-stage bit-width selection algorithm to speech signals with different complexity, some specific intermediate stages may still have a relatively large bit-width deviation. Thus, the bit-width of speech signals with different complexities need to be comprehensively considered, to determine the optimal bit-width selection scheme. 1000 original speech of different keywords are randomly extracted from GSCD, and the proportion of speech signals with different bit-width selection results in each stage are counted and shown in Fig. 2. The error boundary is set to $SQNR_bound = 50dB$. It can be seen that the data bit-width increase gradually when the stage becomes deeper. The first two stages have lower and uniform requirements for the bit-width. From the third stage, the bit-width distribution shows a certain difference, which is because the speech signal of different complexity has different requirements on the bit-width of the FFT calculation at each stage. There are 25% and 65% of the 1000 speech signals can use 14 bit-width and 15 bit-width in stage 9, and the rest 10% signals need use 16 bit-width to ensure accuracy. Therefore, when determining the data bit-width of a specific stage, the bit-width threshold $T(T \in [0, 1])$ should be reasonably determined, e.g., the proportion of the highest data bit-width m for the k_{th} stage is not less than T , then the bit-width of current stage is set as m , otherwise, the bit-width is set as $m - 1$. According to the previous analysis, the resource optimization revenue of the previous stages is greater than that of the latter, and are less error sensitive, thus, the bit-width threshold T_{si} of current stage is reduced stage-by-stage. In

this paper, the T_{si} of the last four stages are set as 0, while the T_{si} of the remaining stages are set as 0.1. Finally, the bit-width is set as [10, 10, 11, 12, 12, 13, 14, 15, 16].

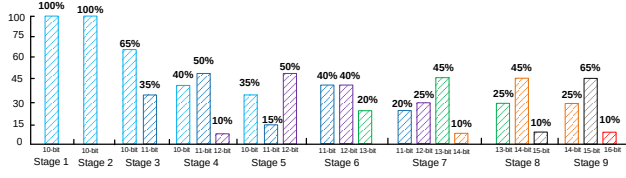


Fig. 2. The proportion of speech signals of radix-2 512-point FFT bit-width selection results at each stage ($SQNR_bound=50dB$).

We utilize the frame-length adaptive MFCC structure, which is proposed in our previous work [27], and the architecture is shown in Fig. 3 (a). It consists of SNR-based Voice Activity Detection (VAD) unit, optimized FFT module based MFCC, and TWN Accelerator. The input speech signal is sampled at 8KHz, and the entire architecture operate on the frames of 40ms with 20ms time step. The VAD unit, which can pre-classify the input speech frames (see Fig. 3(b)), is used to achieve voice activity detecting and redundant frame skipping. When the processor classifies the input speech, a large number of speech frames are redundant, and it is effective to avoid calculating unnecessary frames in speech with different word lengths. When the short-time energy of input speech signal is less than the pre-defined threshold, the unit consider the signal as non-voice input and fill the frames with 0 as the output to TWN accelerator. As shown in Fig. 3 (c), the computation of MFCC can be bypassed with redundant frame padding 0 to save the energy consumption. Through the skip_0 unit in control module, the neural network classifier can also adaptively skip the calculation of redundant frames. What's more, the computation of large amount of 0 weights can also be skipped. With the use of this structure, effective frames which contain the necessary information can be obtained and the computing power consumption can be optimized, as shown in Fig. 3 (d).

IV. SYMMETRIC KERNEL TRAINING BASED TWN

Symmetrical weights have the following two benefits in hardware implementation. First, only one of the two groups of symmetric weights needs to be stored, thus reducing the weight storage. Second, the data flow for symmetric kernel can be optimized for more energy-efficient computing. This section introduces the SKT-based TWN implementation from the algorithm to the hardware, including data reuse under different symmetric modes and the customized data flow.

A. The training process of symmetric TWN model

The proposed TWN model is shown in Fig. 4. Each input speech frame will be extracted into 26 Mel Coefficients through MFCC in frequency domain, and the 1s speech will be processed into 49 frames in time domain. The NN model consists of 4 convolution (Conv) layers and 4 FC layers,

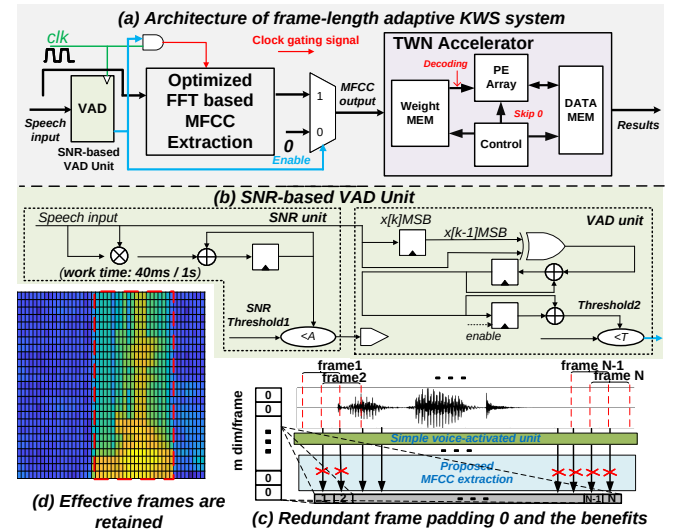


Fig. 3. Frame-length adaptive TWN-based KWS system with SNR-based VAD unit.

with batch normalization (BN) layers and Rectified Linear Unit (ReLU) layers between them. The kernel size of all Conv layers are consistent 3×3 , other hyper-parameters are shown by data in the figure. Compared with NN models utilized by other KWS processors, more FC layers are adopted in our TWN to improve the classification accuracy and the noise robustness. The Conv2 layer and Conv3 layer take up the largest number of operations, accounting for 75.58% and 19.83% respectively, and accompanied by a large amount of memory storage. Therefore, optimizing these two layers can effectively reduce the system hardware overhead. In traditional

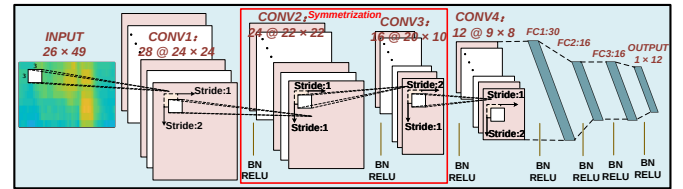


Fig. 4. The proposed TWN model.

TWN model, the threshold δ and scaling factor a are not trainable, and it is impossible to effectively change the threshold to obtain a weight parameter matrix with high sparsity. What's more, the recognition accuracy of the network still has a large gap compared to the floating-point network. Fan et al. pioneered the use of Truncated Gaussian Approximation (TGA) for weight ternary processing [28]. This algorithm focuses on updating thresholds in ternary networks rather than scaling factors. The TGA method can jointly train the quantizer thresholds and weights, to minimize the accuracy drop caused by model compression. However, the sparsity in TGA is not controllable. In this work, we introduce an adjustable sparsity parameter S_l to scale the threshold δ of the ternary weights to control the weight sparsity. As shown in Fig. 5 (a), the ternary weight update is divided into two steps. First, the threshold of the ternary weight is calculated according to the weight of the current layer l and the sparsity

parameter S_l . Second, the weight is updated according to the newly updated threshold. After the sparsity-controllable quantization scheme, the sparsity of the weight is increased to 33.6%, $6.5\times$ higher than the full-precision model.

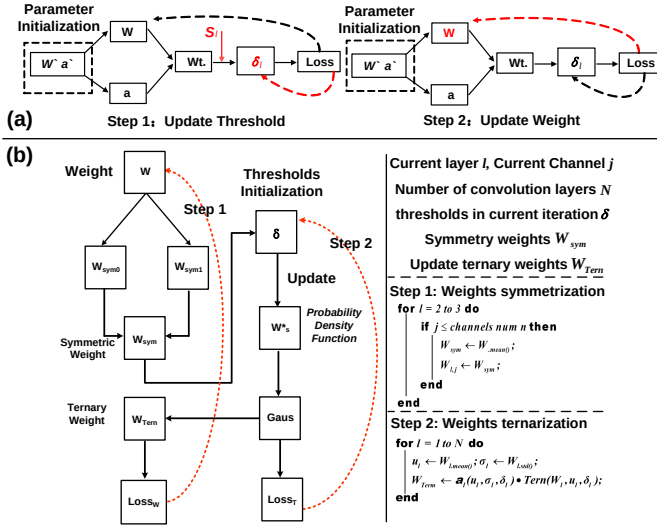


Fig. 5. (a) Sparsity-controllable ternary weight quantization scheme. (b) Symmetric kernel training process.

In order to further utilize the sparsity-controllable compression scheme and reduce the computation and storage overhead of TWN, symmetric kernel training (SKT) is proposed in this work, and its overall training method is shown in Fig 5 (b). At the beginning of training (0^{th} epoch), the weight, threshold and scaling factor are initialized based on a pre-trained neural network. At t^{th} epoch, the weights and thresholds are updated based on the structure of the last epoch. Two steps are processed in the overall training, first weight symmetrized and then ternarized. Two operations can be used to compute symmetrical weights, maximum or average. Through experiments and comparison, averaging the weights at symmetrical positions can have obtain a higher accuracy. Then, the symmetrical weights are ternary quantized. The ternary weight threshold is calculated according to mean value μ_l , variance value σ_l of all weights in each Conv layer and sparsity parameter s_l . The ternary weight is updated according to the newly computed threshold and the ternary function [28]. The scaling factor a in Conv layers and FC layers are extracted and combined to BN layer, thus the multiplication can be processed with NAND gate and adder. By adopting the proposed SKT method, the weight in TWN can be significantly compressed, with the accuracy loss less than 1% under different background noises.

B. Symmetric modes and data reuse in SKT based TWN

Increasing repeated weights has become an effective way to improve the energy efficiency of DNN accelerators [29]. In TWN, the phenomenon of repeated weights becomes more significant, in 3×3 convolution kernel, 33.3% repeated weights occur when the weight distribution satisfies the uniform distribution. To further utilize the repetitive weights, we propose SKT, which enables efficient weight compression and data

reuse. Three different symmetric modes can be obtained by SKT, including transverse symmetry, longitudinal symmetry and diagonal symmetry, shown in Fig. 6. For each symmetric mode, there are single weight symmetry and group weight symmetry. Single weight symmetry means that only two weights at the corresponding position is equal, while group weight symmetry means that multiple consecutive weights are the same. As single weight symmetry can only generate little data reuse in hardware implementation, we mainly focus on group weight symmetry in this work. In the TWN kernel without SKT, 34.59% of the single weight and 6.17% of the group weight are intrinsically symmetric, and the recognition accuracy is 92.2%. After utilizing SKT in Conv2 and Conv3 layer, the symmetric rate of group weight and single weight are increased from 6.17% to 66.46% and 34.59% to 89.48% respectively. The recognition accuracy is reduced by 0.5%. Utilizing SKT in Conv1 layer simultaneously will not improve the symmetry rate significantly, while the accuracy loss is 4.3%. Since the Conv2 and Conv3 are also the layers with the largest number of parameters and operations, we adopt the SKT methods in these two layers.

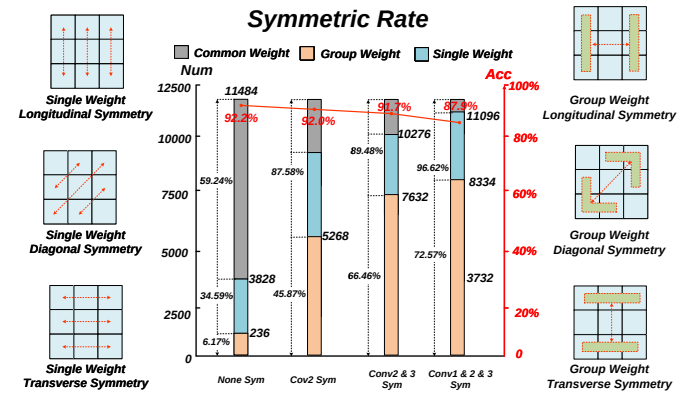


Fig. 6. Symmetric modes and symmetric rate in TWN.

The data reuse corresponding to different symmetric modes are also different. Since diagonal symmetry has symmetrical convolution weights in both the time domain and the frequency domain, it is difficult to generate a large amount of reusable data in the convolution process, so it is not suitable for deployment in hardware implementation. For transverse symmetric and longitudinal symmetric kernels, they are easier to generate data multiplexing, and their corresponding computational data flow is shown in Fig. 7. For transverse symmetric kernels, when the frames are sequentially input to processing elements (PE) through the FIFO, they will be convolved with the corresponding weights. The weight reuse occurs in frequency domain, the weights of first row can be explicitly reused with the weights of third row in one convolution operation. What's more, after the kernel move twice in frequency domain, the previous calculated output can be reused. For a longitudinally symmetric kernel, output reusing occurs only when the filter moves in time domain, i.e. the different input frames, which makes it harder to achieve. Besides, through our experiment, the transverse symmetric can achieve the highest accuracy

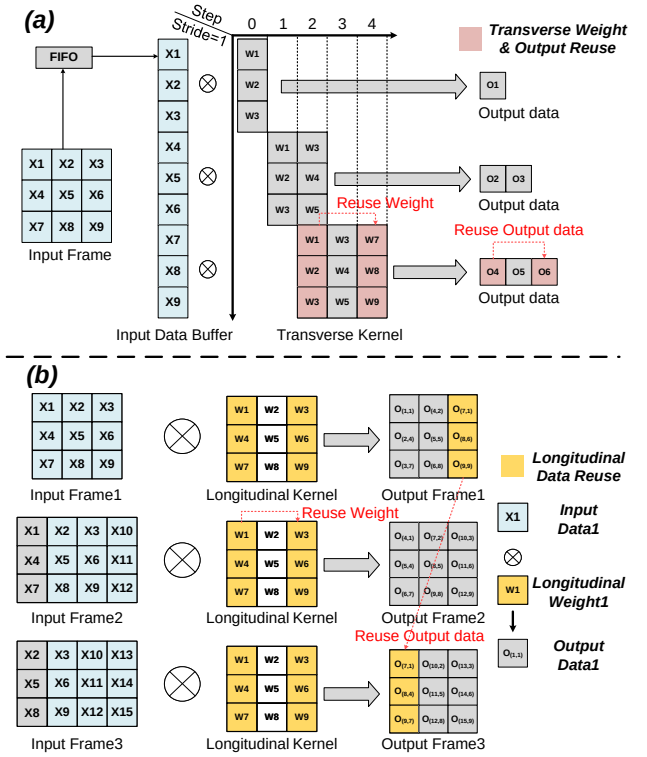


Fig. 7. Data reuse in (a) transverse symmetric kernel (b) longitudinal symmetric kernel.

(91.9%), which is 0.7% higher than longitudinal symmetric and 1.2% higher than diagonal symmetry. Therefore, we adopt transverse symmetry for all symmetric kernels.

C. The Architecture and Data Flow of TWN Accelerator

For the efficient hardware implemen of the symmetric convolution kernel, we propose an optimized TWN accelerator, the overall structure is shown in Fig. 8 (a). It mainly consists of the computing array with 30 PEs, the main control unit to schedule the load and store of the weight and data, the weight decode unit to decompress the encoded weight, the post process unit to calculate BN and ReLU layers and on-chip SRAM for data/weight/parameter storage.

The sparsity-controllable and symmetric training method of TWN enable weight compression coding to effectively reduce the weight storage. This work utilize the Huffman-Run length Coding (H-RLC), which is obtained by combining Run Length Coding (RLC) and Huffman Coding, to achieve a better balance between compression rate and decoding complexity. As shown in Fig. 8 (b), *Flag* is used to indicate whether the weight is encoded, *Weight* is the 2-bit weight value, and *Coding Table* is used to indicate the number of repeated weights. The Coding Table is encoded by Huffman coding, it can encode the repetition number of RLC more efficiently. With this approach, the weight storage can be reduced by 2.74KB. Fig. 8 (c) shows the H-RLC decoding unit, which sequentially outputs the identical repetition weights and weight valid signals through a finite state machine (FSM) and counter.

The PE array adopts the serial pipeline computing method to process the convolution layer by layer. Each PE can be

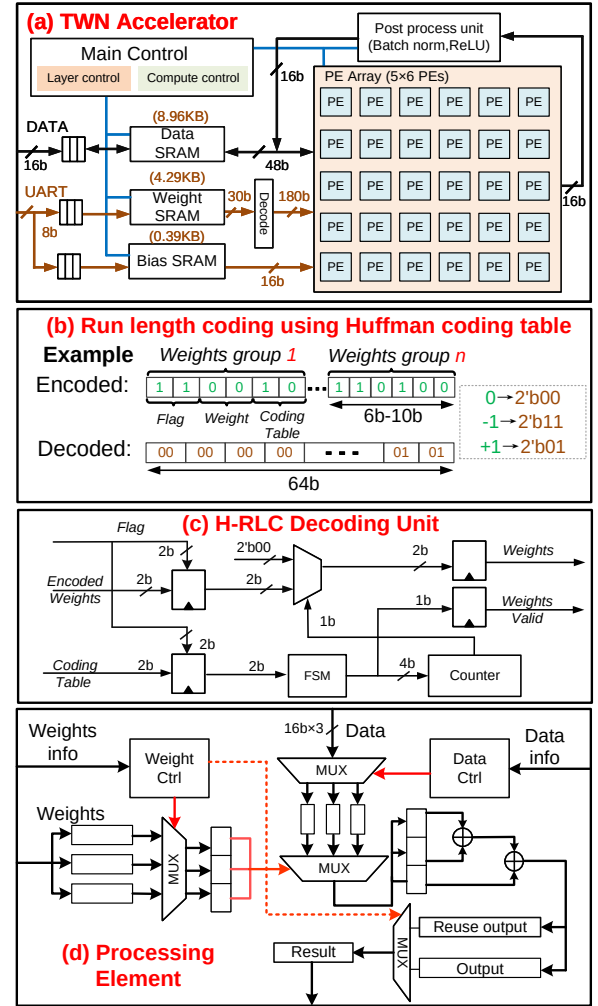


Fig. 8. (a) Architecture of TWN Accelerator. (b) Huffman-Run length Coding. (c) Structure of H-RLC Decoding Unit. (d) Structure of Processing Element.

controlled by the main control unit. “Weights info” is a 1-bit flag signal stored right after each 3×3 convolution kernel, which indicates whether the kernel is symmetric. For the symmetric kernels, weight control unit will control the PEs to store the reuse weights and outputs, shown in Fig. 8 (d). The memory control unit will only load one group of symmetric weights, thus improving the memory access efficiency. The SKT method brings an additional 1.55KB of weight compression. Compared to the TWN accelerator without SKT, the power consumption of SKT based optimized TWN accelerator can be reduced by 18.6%.

Fig. 9 shows the customized data flow with and without symmetry. First, three weights and data are processed in one PE, and the 3×3 convolution is processed by a set of 3 PEs in one cycle. Then, in the $t+1$ clock cycle, the weight originally input to three PEs remains unchanged, the input data to PE₁ at cycle t is transmitted to PE₀, the input data of PE₂ is transmitted to PE₁, and PE₂ inputs the new data to process a new convolution. In the symmetric convolution kernel, for a convolution calculation with a frequency domain step of 1, a repeated partial sum will appear in PE₀ every two times the input data is moved. Therefore, PE₀ is not processed in the $t+2$ cycle, the output partial sum is directly the results stored

in the "resue output" register of PE_2 in cycle t .

Data tiling is the standard methodology in DNN processor [14]. In this work, in order to adapt to the designed PE structure and symmetrical data flow, the PEs in the PE array are divided into groups of three. 30 PEs are divided into 10 groups to process 10 output channels simultaneously, and the data is broadcast to these 10 groups of PEs. Input channels will be processed sequentially in a group of PEs. Using this time-sharing multiplexing method, the convolution of large input channel and output channel can be processed more effectively.

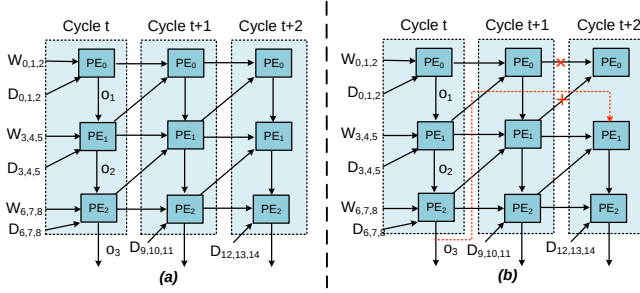


Fig. 9. (a) Data flow in PE array without Symmetry. (b) Data flow in PE array with Symmetry.

V. ERROR INTER-COMPENSATION APPROXIMATE ADDITION TREE WITH CGP-BASED ERROR INTER-COMPENSATION SCHEME

Since multiplications in convolution are eliminated in TWN, addition operation is revealed to have significant impact on the computing power dissipation. In this section, we propose an energy efficient approximate addition tree to further reduce the computing overhead. An error inter-compensation scheme is proposed to realize the optimal compensation between different approximate adders in the approximate addition tree.

A. Error Inter-Compensation Approximate Addition Tree

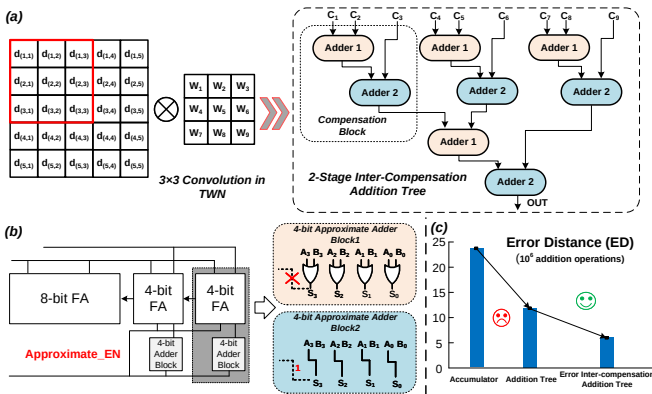


Fig. 10. (a) 2-stage inter-compensation addition tree in TWN convolution. (b) 16-bit adder with 2 approximate adder blocks. (c) Error comparison of accumulator and addition tree.

As shown in Fig. 10 (a), the 3×3 convolution kernel is organized as a 2-stage addition tree. In each stage, a compensation block is composed of 2 different approximate adders,

and processed within one cycle. These two approximate adders are constructed to compensate the error of each other. A 32-bit Lower-part-OR Adder (LOA) [30] is used to accumulate 10^6 random data. The result is evaluated by Error Distance (ED), which is defined as the absolute value of the difference between exact result and approximate result, shown in Fig. 10 (c). The addition tree has a lower ED than the traditional accumulator, while the error inter-compensation addition tree has the lowest. Fig. 10 (b) shows the 16-bit approximate adders, Full adders (FA) are used in the high bits and the low 8-bit can be configured as FAs or approximate adders every 4-bit. "Approximate EN" is used to configure accurate or approximate modes. 4-bit approximate adder block 1 is composed of 4 OR-gate approximate adder with no internal carry and external carry out. The approximate adder block 2 is the compensation approximate adder, composed of 4 wires directly connecting input "A" to output "Sum" with a fixed external carry out "1". It is the iterative result of our proposed compensation scheme, also called as the compensation adder (CA). Compared with FA, OR-gate approximate adder and CA have fewer transistors and shorter delays. We evaluated the performance of TWN accelerator deployed with exact and approximate adders at the circuit level, on an industrial TSMC 22nm process technology with the voltage of 0.54V and clock frequency of 250 KHz. The experiment results are shown in Fig. 11. The approximate addition tree can bring a power reduction of 17.3% in TWN accelerator, with the accuracy loss of 0.5% (Clean) $\sim$ 1.3% (5dB).

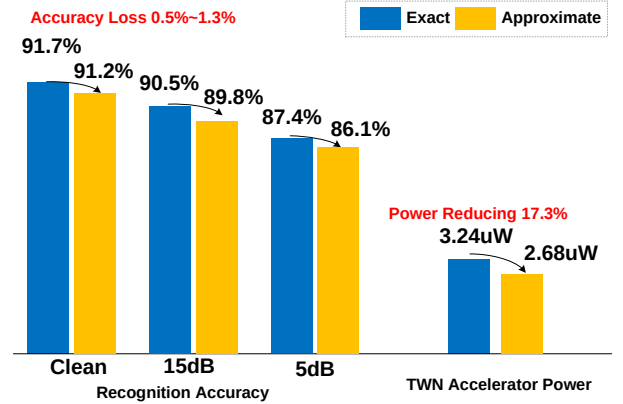


Fig. 11. Approximate effect on recognition accuracy and power consumption (software baseline).

B. CGP-based Error Inter-Compensation Scheme

In DNN, error of uniform distribution is more likely to compensate each other during the calculation and have a higher accuracy. We originate a Cartesian Genetic Programming (CGP) based error inter-compensation scheme to compensate the error of a certain approximate structure. CGP is a heuristic automatic design method. Through this scheme, the error distribution of the compensated heterogeneous addition tree presents a uniform distribution of positive and negative errors and achieves an optimization of calculation accuracy and hardware efficiency.

OR-gate adder has a simple structure which is suitable to be chosen as the baseline approximate adder. The error distribution of 4-bit OR-gate adder block is obtained through simulation and then be input to the fitness function of CGP as a parameter. The target error characteristics is also computed as the searching parameter of CGP. Finally, the compensation adders are grouped into the error inter-compensation addition tree and deployed on the TWN accelerator. The error distribution of the addition tree becomes more uniform after applying the error inter-compensation scheme.

In CGP, each chromosome (i.e. candidate result in iteration) is a two-dimensional graph structure, and the function of each node can be modelled as Boolean functions, it has been proven to generate high-quality approximate circuits [31]. All generated candidate circuits are evaluated by a fitness function, which can be expressed as the mismatch between generated circuit accuracy and target accuracy. The fitness function of our compensation scheme is shown in Equation (3). Sum of error distances (SED) characterizes the overall error characteristics of the approximate adder. The target is a 4-bit approximate adder block, so there are a total of 256 input combinations. As shown in Equation (4), S represents the calculation result of the candidate approximate adder, and S' represents the exact calculation result, i and j are each input situation of two 4-bit addends. SED_{ORA} represents the error of the OR-gate approximate adder obtained by the previous exhaustive simulation, which is used to control the overall error of generated compensation approximate adder close to the given OR-gate adder. SED_{CA} represents the error of the compensation approximate adder. The SED of compensation adder is set to be twice as the SED of OR-gate adder, due to the intrinsic error of two addends.

$$Fitness = SED_{CA} - 2 \times SED_{ORA} + Cost \quad (3)$$

$$Error_{i,j} = S_{i,j} - S'_{i,j} \quad (4)$$

$$SED = \sum_{i=1}^{16} \sum_{j=1}^{16} |Error_{i,j}|$$

$$Cost = \begin{cases} 0, & \forall (Error_{i,j} \geq 0) \\ \infty, & \exists (Error_{i,j} < 0) \end{cases} \quad (5)$$

Notice that $Cost$ is the cost function we introduced to constrain the generated approximate circuit, shown in Equation (5). Only when the error of the generated approximate circuit is all positive, the $Cost$ is 0. It ensures that the CGP evolves in the direction of the approximate adder whose error distribution is all positive to meet the compensation requirements. The extremely simple structure of the compensation adder generated by the error inter-compensation scheme is shown in Fig. 10 (b).

VI. CHIP IMPLEMENTATION AND EVALUATION RESULTS

Fabricated in TSMC 22-nm ultra-low-leakage (ULL) process technology with HVT transistors, the prototype tapeout results of the proposed TWN KWS processor is shown in Fig. 12. To evaluate the power consumption and recognition accuracy, the set-up test platform mainly includes power

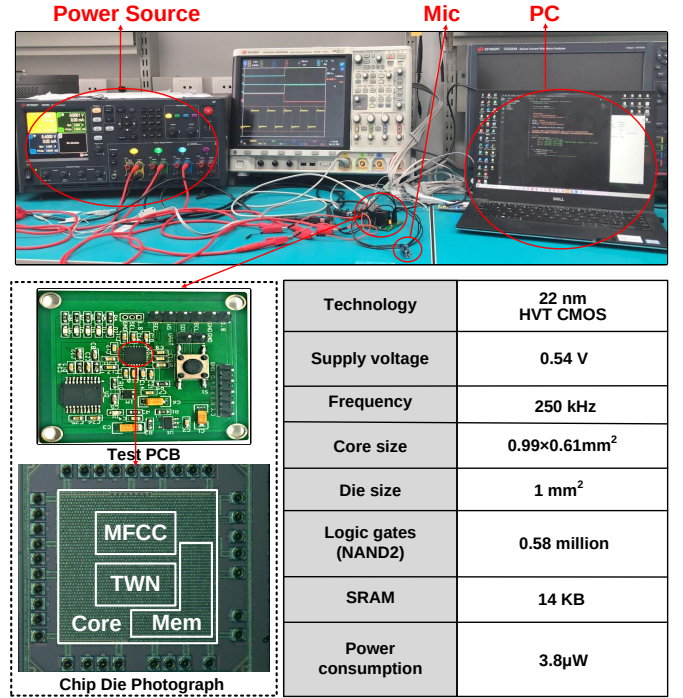


Fig. 12. Test platform and results of the chip

supply, oscilloscope, microphone, PC and a testing PCB board with the chip on a socket. PC transmits the weight parameters required by the chip to the chip through UART, and then the external voice signal is collected by the commonly used external digital silicon microphone SPH0645, and the voice signal is transmitted to the chip through I2S. We conducted tests on a total of 30 chips to evaluate their performance. The average power consumption ranged from 3.8μW to 150μW, with 250KHz to 10MHz clock frequency from 0.54V to 0.9V. The chip testing result is listed in Fig. 12, the prototype system is functional with supply voltage of 0.54V and clock frequency of 250 KHz. The chip achieves a minimum power consumption of 3.8 μW, with an on-chip 14 KB memory and a core area of 0.99×0.61mm².

A. Accuracy and Power Measurement

The feature extraction module operates on the 20ms time step at 8kHz sample rate. It can be dynamically reconfigured to padding 0 for redundant frames. We use the Google speech command dataset (GSCD) as the training and validation set, which contains 105K 1-second-long audio clips of 35 keywords in the dataset. Table I shows the chip testing accuracy under different noise scenarios. A total of 11 different background noise types, which are all from NoiseX-92 dataset, and four different SNR intensities of 5dB, 10dB, 15dB and Clean were used for testing. Under various noise types, the chip has a high noise robustness, reaching the accuracy of over 85% under 5dB, over 88% under 10dB and over 89.1% under 15dB. Compared with the software baseline, there exists slightly accuracy losses, within 1%, in chip testing result, which is caused by the subtle differences between the test environment and the software simulation environment.

TABLE I
CHIP TESTING ACCURACY UNDER DIFFERENT NOISE SCENARIOS

Noise Type*	15dB	10dB	5dB
Babble	89.7%	88.1%	85.3%
Buccaneer1	89.9%	88.4%	85.4%
Buccaneer2	89.3%	88.6%	85.5%
Destroyerops	89.4%	88.5%	85.0%
Factory1	89.6%	88.0%	85.4%
Factory2	90.2%	88.7%	85.2%
M109	89.2%	88.8%	85.1%
Machinegun	89.2%	88.8%	85.0%
Pink	89.4%	88.1%	86.4%
Volvo	89.1%	88.2%	86.4%
White	89.7%	88.6%	85.2%

(*The accuracy under clean is 90.6%. All noises are from NoiseX-92.)

B. Analysis of Area and Power breakdown

The area and power breakdowns of the processor and TWN accelerator are illustrated in Fig. 13 and Fig. 14, which are obtained by post layout simulations with supply voltage of 0.54V. It contains TWN accelerator module, MFCC module, Top Control module, and some other I/O interfaces. Among them, the TWN accelerator module occupies the largest proportion, accounting for 58% of the total area and 70.42% of the total power consumption, respectively. Due to the stage-by-stage bit-width selection algorithm of FFT, the power consumption of MFCC module is significantly reduced. In the processor, leakage power and dynamic power consumption accounted for 42.7% and 57.2% of the total power consumption, respectively.

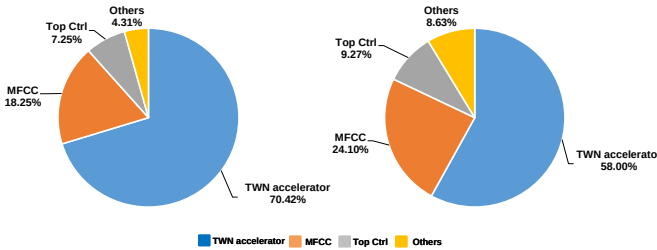


Fig. 13. Power breakdown (left) and Area breakdown (right) of the processor

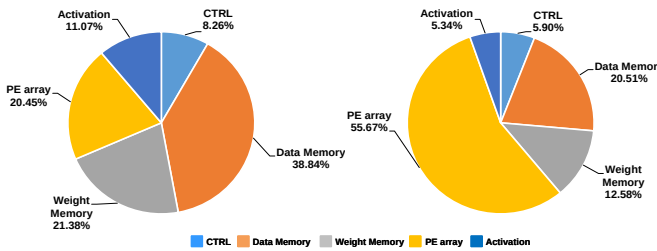


Fig. 14. Power breakdown (left) and Area breakdown (right) of TWN accelerator

TWN accelerator is divided into control unit, data memory, weight memory, PE array and non-linear activation unit. Among them, PE array takes up the largest proportion of area,

reaching 55.67%, while data memory consumes the largest power consumption of 38.84%. Several techniques are used to cut down the power consumption: First, the complex control and multiplexers enables the reuse of output partial sum and weights in symmetric convolution kernel during calculation. Second, the design of inter-compensation approximate addition tree reduces the addition power consumption. Since the data memory needs to store the calculation results in each layer, and the weight memory is compressed by SKT, the area and power consumption of weight memory is much lower than data memory. Leakage power and dynamic power accounted for 40.9% and 59.1% of the total power consumption of the TWN accelerator.

C. Comparisons With State-of-the-art Works

Table II lists the comparison between this work and state-of-the-art KWS chips. Compared with [1], [5] and [11], our work can support the background noise from 5dB~clean, while reaching the lowest power consumption. Compared with [14], our work can support more keywords (12 classes with silence and unknown) recognition and shorter response time. The high noise robustness in this work is benefited from the using of deeper network architecture with more layers and finer-grained weight adjustment capability brought by ternary weights. The proposed TWN module has the largest operation per frame compare to the other works. Based on the more complex DNN topology which is more error tolerant than other works, the proposed TWN can be trained using a dataset that incorporates speech feature extractions from each different background noise. This also increases the noise robustness of this work. By adopting the software and hardware co-optimized SKT based TWN accelerator with bit-width optimized FFT and approximate addition tree, our KWS chip can achieve the highest normalized energy efficiency in the work of 10 keywords, which is $2.79\times$ higher than previous work [5].

VII. CONCLUSION

This paper proposed a 5dB~Clean wide range of background noises supported and low power ternary-weight keyword spotting processor with end-to-end optimized hardware deployment. To achieve the balance between high accuracy and ultra-low power, the bit-width of each stage in FFT can be accurately selected. In keywords classification, we propose the symmetric kernel training to compress the weight and calculations in TWN. The architecture of TWN accelerator and data flow is then optimized to utilize the weight/output reuse of the symmetrical TWN. By using error inter-compensation approximate addition tree, the computing power can be further reduced in TWN accelerator. The processor is fabricated on 22-nm CMOS technology, with the ultra-low power of $3.8\mu\text{W}$. Compared to the state-of-the-art, the proposed design performs $2.79\times$ better referring normalized power efficiency.

REFERENCES

- [1] J.S.P. Giraldo and M. Verhelst, "Laika: A $5\mu\text{W}$ Programmable LSTM Accelerator for Always-on Keyword Spotting in 65nm CMOS," in *IEEE 44th European Solid State Circuits Conference (ESSCIRC)*, 2018, pp. 166-169.

TABLE II
COMPARISONS WITH THE STATE-OF-THE-ART LOW POWER KEYWORDS RECOGNITION PROCESSORS

	This work	ESSCIRC'18 [1]	JSSC'20 [5]	ISSCC'20 [14]	ISSCC'22 [11]
Technology	22nm	65nm	65nm	28nm	65nm
DNN Architecture	TWN	LSTM	LSTM	DSCNN	RNN
Bit width (Data/Weight)	16/2-bit	NA	4(8)/8-bit	1/1-bit	14/8-bit
Ops./Frame	4930K	43K	53K	150K	48K
Frequency (KHz)	250	250	250	40	250
Latency (ms)	16	16	16	64	12.4
Voltage (V)	0.54	0.575	0.6	0.41	0.5/0.75
Layout Area (mm²)	0.60	1.04	2.56	0.23	2.03
Memory (KB)	14	32	65	2	27
Power (μW)	3.8	5*	10.6	0.51	13.6**
Normalized energy efficiency***	$2.79 \times (10 \text{ words})$	$2.12 \times (4 \text{ words})$	$1 \times (10 \text{ words})$	$5.20 \times (1 \sim 2 \text{ words})$	$1.01 \times (10 \text{ words})$
Background noise support	SNR: 5dB~Clean	Clean	Clean	SNR: 5dB~Clean	Clean
Recognition accuracy	(10 words) Clean: 90.6% 5dB: 85.4%	(4 words) Clean: 91.2%	(10 words) Clean: 90.9%	(1 word) Clean: 98.0% 5dB: 90.7%	(10 words) Clean: 86.0%

(*5 μ W does not include the power consumption of MFCC. **13.6 μ W is the power consumption of digital back-end. ***Normalized energy efficiency is calculated by (Latency $\times$ Power)/Baseline (the baseline is reported in JSSC'20 [5]).)

- [2] G. Chen, C. Parada and G. Heigold, "Small-footprint keyword spotting using deep neural networks," in *IEEE International Conference on Acoustics, Speech and Signal Processing (ICASSP)*, 2014, pp. 4087-4091.
- [3] J.S.P. Giraldo, S. Lauwereins, K. Badami, H. Van Hamme and M. Verhelst, "18 μ W SoC for near-microphone Keyword Spotting and Speaker Verification," in *Symposium on VLSI Circuits*, 2019, pp. C52-C53.
- [4] S. Yin et al., "A 141 μ W, 2.46 PJ/Neuron Binarized Convolutional Neural Network Based Self-Learning Speech Recognition Processor in 28NM CMOS," in *IEEE Symposium on VLSI Circuits*, 2018, pp. 139-140.
- [5] J. S. P. Giraldo, S. Lauwereins, K. Badami and M. Verhelst, "Vocell: A 65-nm Speech-Triggered Wake-Up SoC for 10- μ W Keyword Spotting and Speaker Verification," *IEEE Journal of Solid-State Circuits*, vol. 55, no. 4, pp. 868-878, April 2020.
- [6] M. Shah et al., "A fixed-point neural network architecture for speech applications on resource constrained hardware," in *JSPS*, vol. 90, no. 5, pp. 727-741, 2018.
- [7] D. Wang, S. J. Kim, M. Yang, A. A. Lazar and M. Seok, "9.9 A Background-Noise and Process-Variation-Tolerant 109nW Acoustic Feature Extractor Based on Spike-Domain Divisive-Energy Normalization for an Always-On Keyword Spotting Device," in *IEEE International Solid-State Circuits Conference (ISSCC)*, 2021, pp. 160-162.
- [8] H. Dbouk, S. K. Gonugondla, C. Sakr and N. R. Shanbhag, "A 0.44- μ J/dec, 39.9- μ s/dec, Recurrent Attention In-Memory Processor for Keyword Spotting," *IEEE Journal of Solid-State Circuits*, vol. 56, no. 7, pp. 2234-2244, July 2021.
- [9] Z. Wang et al., "A 148-nW Reconfigurable Event-Driven Intelligent Wake-Up System for AIoT Nodes Using an Asynchronous Pulse-Based Feature Extractor and a Convolutional Neural Network," *IEEE Journal of Solid-State Circuits*, vol. 56, no. 11, pp. 3274-3288, Nov. 2021.
- [10] L. Zhu, W. Shan, J. Xu and Y. Lu, "AAD-KWS: a sub- μ W keyword spotting chip with a zero-cost, acoustic activity detector from a 170nW MFCC feature extractor in 28nm CMOS," in *IEEE 47th European Solid State Circuits Conference (ESSCIRC)*, 2021, pp. 99-102.
- [11] K. Kim et al., "A 23 μ W Solar-Powered Keyword-Spotting ASIC with Ring-Oscillator-Based Time-Domain Feature Extraction," in *IEEE International Solid-State Circuits Conference (ISSCC)*, 2022, pp. 1-3.
- [12] M. F. F. Khan, M. M. Kamani, M. Mahdavi and V. Narayanan, "Learning to Quantize Deep Neural Networks: A Competitive-Collaborative Approach," in *57th ACM/IEEE Design Automation Conference (DAC)*, 2020, pp. 1-6.
- [13] S. Yin et al., "An Energy-Efficient Reconfigurable Processor for Binary- and Ternary-Weight Neural Networks With Flexible Data Bit Width," *IEEE Journal of Solid-State Circuits*, vol. 54, no. 4, pp. 1120-1136, April 2019.
- [14] W. Shan et al., "14.1 A 510nW 0.41V Low-Memory Low-Computation Keyword-Spotting Chip Using Serial FFT-Based MFCC and Binarized Depthwise Separable Convolutional Neural Network in 28nm CMOS," in *IEEE International Solid-State Circuits Conference-ISSCC*, 2020, pp. 230-232.
- [15] D. Wang, C.T. Lin, G.K. Chen, P. Knag, R.K. Krishnamurthy and M. Seok, "DIMC: 2219TOPS/W 2569F2/b Digital In-Memory Computing Macro in 28nm Based on Approximate Arithmetic Hardware," in *IEEE International Solid-State Circuits Conference (ISSCC)*, 2022, pp. 266-268.
- [16] Y. Wang et al., "A 28nm 27.5TOPS/W Approximate-Computing-Based Transformer Processor with Asymptotic Sparsity Speculating and Out-of-Order Computing," in *IEEE International Solid-State Circuits Conference (ISSCC)*, 2022, pp. 1-3.
- [17] Z.G. Tasoulas, G. Zervakis, I. Anagnostopoulos, H. Amrouch and J. Henkel, "Weight-Oriented Approximation for Energy-Efficient Neural Network Inference Accelerators," *IEEE Transactions on Circuits and Systems I: Regular Papers*, vol. 67, no. 12, pp. 4670-4683, Dec. 2020.
- [18] H. Jiang et al., "Approximate Arithmetic Circuits: A Survey, Characterization, and Recent Applications," *Proceedings of the IEEE*, vol. 108, no. 12, pp. 2108-2135.
- [19] W. Liu, F. Lombardi and M. Shulte, "A Retrospective and Prospective View of Approximate Computing," *Proceedings of the IEEE*, vol. 108, no. 3, pp. 394-399, March 2020.
- [20] H. A. F. Almurib, T. N. Kumar and F. Lombardi, "Inexact designs for approximate low power addition by cell replacement," in *Design, Automation & Test in Europe Conference & Exhibition (DATE)*, 2016, pp. 660-665.
- [21] W. Liu, J. Xu, D. Wang, C. Wang, P. Montuschi and F. Lombardi, "Design and Evaluation of Approximate Logarithmic Multipliers for Low Power Error-Tolerant Applications," *IEEE Transactions on Circuits and Systems I: Regular Papers*, vol. 65, no. 9, pp. 2856-2868, Sept. 2018.
- [22] D. Esposito, A. G. M. Strollo, E. Napoli, D. De Caro and N. Petra, "Approximate Multipliers Based on New Approximate Compressors," *IEEE Transactions on Circuits and Systems I: Regular Papers*, vol. 65, no. 12, pp. 4169-4182, Dec. 2018.
- [23] B. Liu et al., "A 22nm, 10.8 μ W/15.1 μ W Dual Computing Modes High Power-Performance-Area Efficiency Domained Background Noise Aware Keyword-Spotting Processor," *IEEE Transactions on Circuits and Systems I: Regular Papers*, vol. 67, no. 12, pp. 4733-4746, Dec. 2020.
- [24] P.Y. Chuang, P.Y. Tan, C.W. Wu and J.M. Lu, "A 90nm 103.14 TOPS/W Binary-Weight Spiking Neural Network CMOS ASIC for Real-Time Object Classification," in *57th ACM/IEEE Design Automation Conference (DAC)*, 2020, pp. 1-6.
- [25] Li, Fengfu, Bo Zhang, and Bin Liu. "Ternary weight networks," May. 2016, arXiv:1605.04711. [Online]. Available: <https://arxiv.org/abs/1605.04711>
- [26] W. Liu, Q. Liao, F. Qiao, W. Xia, C. Wang and F. Lombardi, "Approximate Designs for Fast Fourier Transform (FFT) With Application to Speech Recognition," *IEEE Transactions on Circuits and Systems I: Regular Papers*, vol. 66, no. 12, pp. 4727-4739, Dec. 2019.
- [27] B. Liu, X. Ding, H. Cai, W. Zhu, Z. Wang, W. Liu and J. Yang, "Precision Adaptive MFCC Based on R2SDF-FFT and Approximate Computing for Low-Power Speech Keywords Recognition," *IEEE Circuits and Systems Magazine*, vol. 21, no. 4, pp. 24-39, Fourthquarter 2021.
- [28] Z. He and D. Fan, "Simultaneously Optimizing Weight and Quantizer of Ternary Neural Network Using Truncated Gaussian Approximation,"

in *IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2019, pp. 11430-11438.

- [29] H. Mo et al., "A 12.1 TOPS/W Quantized Network Acceleration Processor With Effective-Weight-Based Convolution and Error-Compensation-Based Prediction," *IEEE Journal of Solid-State Circuits*, vol. 57, no. 5, pp. 1542-1557, May 2022.
- [30] H. R. Mahdiani, A. Ahmadi, S. M. Fakhraie and C. Lucas, "Bio-Inspired Imprecise Computational Blocks for Efficient VLSI Implementation of Soft-Computing Applications," *IEEE Transactions on Circuits and Systems I: Regular Papers*, vol. 57, no. 4, pp. 850-862, April 2010.
- [31] V. Mrazek, L. Sekanina and Z. Vasicek, "Libraries of Approximate Circuits: Automated Design and Application in CNN Accelerators," *IEEE Journal on Emerging and Selected Topics in Circuits and Systems*, vol. 10, no. 4, pp. 406-418, Dec. 2020.
- [32] Damsgaard, Hans Jakob, Aleksandr Ometov, and Jari Nurmi. "Approximation Opportunities in Edge Computing Hardware: A Systematic Literature Review." *ACM Computing Surveys* 55.12 (2023): 1-49.



Bo Liu (Member, IEEE) received the B.S. and Ph.D. degrees in electronic science and engineering from Southeast University, Nanjing, China, in 2006 and 2013 respectively. He is currently an Associate Professor with National ASIC system Engineering Research Center, School of Integrated Circuits, Southeast University. His research interests include chip architecture design, reconfigurable computing, approximate computing, and related VLSI designs. He has authored or coauthored more than 50 scientific papers in the above research fields, and holds

one U.S. patent and more than 30 Chinese patents. His research is supported by the National Natural Science Foundation, National Science and Technology Major Project, and National Key R&D Program. He has authored or coauthored two book chapters and more than 30 scientific papers, including *Nature Electronics*, *IEEE TCAS-I/II*, *TCAD* and *IEEE Circuits and Systems Magazine*.



Na Xie received the B.S. degree in Electronics and Information technology from Sun Yat-Sen University, Guangzhou, China, in 2021. He is currently pursuing the M.S. degree in Integrated Circuit Engineering from Southeast University, Nanjing, China. His current research interests include Neural Network accelerator design and approximate computing.



Renyuan Zhang received the Bachelor degree in Electronic Science and Technology from Hefei University of Technology, Hefei, China, in 2020. He is currently pursuing the Master degree in Integrated Circuit Engineering from Southeast University, Nanjing, China. His current research interests include DNN accelerator and approximate computing.



Haichuan Yang received the B.S. degree in microelectronics science and engineering from the Nanjing University of Posts and Telecommunications, Nanjing, China, in 2019. And he received the M.S. degree in integrated circuit engineering with Southeast University, Nanjing, China, in 2022. His current research interests include neural networks and approximate computing.



Ziyu Wang received the B.S. degree in Microelectronic science and engineering from Anhui University, Hefei, China, in 2020. He is currently pursuing the M.S. degree in Integrated Circuit Engineering from Southeast University, Nanjing, China. His current research interests include digital application specific integrated circuit design and low voltage circuits.



Deliang Fan (Member, IEEE) is currently an Associate Professor in the Department of Electrical and Computer Engineering, Johns Hopkins University, Baltimore, MD, USA. He received his M.S. and Ph.D. degrees in Electrical and Computer Engineering from Purdue University, West Lafayette, IN, USA, in 2012 and 2015, respectively. Dr. Fan's primary research interests include Machine Learning Circuits and Algorithm at Edge, In-Memory Computing Circuits and Architecture, Adversarial and Trustworthy AI System. He has authored and co-authored around 160+ peer-reviewed international journal/conference papers in those areas. He is the receipt of NSF Career award, best IP paper award of DATE 2022, best paper award of GLSVLSI 2019, ISVLSI 2018 and ISVLSI 2017. His research works were also nominated as best paper candidate of ASPDAC 2019, ISQED 2019. He is the program chair of ISQED 2023, iMACAW@DAC 2022/2023. He is the technical area chair of DAC 2021, GLSVLSI 2019-2022, ISQED 2019-2022, and the financial chair of ISVLSI 2019. More details are in his research website: <https://www.ece.jhu.edu/dfan/>.



Zhen Wang received the B.S. degree in radio from Southeast University in 2002 and the M.S. and Ph.D. degrees in electronic science and engineering from Southeast University in 2005 and 2018, respectively. He is currently working with Nanjing Prochip Electronics Technology Company Ltd. His research interests include calculation in memory, low power circuit, and the AI voice circuit.



Weiqiang Liu (Senior Member, IEEE) is currently a Professor and the Vice Dean of College of Electronic and Information Engineering, Nanjing University of Aeronautics and Astronautics (NUAA), Nanjing, China. He received the B.S. degree in Information Engineering from NUAA and the Ph.D. degree in Electronic Engineering from Queens University Belfast (QUB), Belfast, United Kingdom, in 2006 and 2012, respectively. In Dec. 2013, he joined the College of Electronic and Information Engineering, NUAA. He has served as a Guest Editor of Pro-

ceedings of the IEEE and Associate Editors of IEEE TCAS-I, TETC, TC. He received the prestigious Excellent Young Scientists Fund from NSFC in 2020. His research interests include emerging technologies in computing systems, computer arithmetic, hardware security and VLSI design for digital signal processing and cryptography. He has published one research book and over 130 leading journal and conference papers.



Hao Cai (Senior Member, IEEE) received the master degree in electrical engineering from Lund University, Sweden, in 2009, and the Ph.D. degree in electrical engineering from TELECOM ParisTech, Paris, France, in 2013. From 2013 to 2017, he was with Université Paris-Saclay, Paris, France, where he worked on reliable low-power integrated circuits design under the European EUREKA program CATRENE-RELY and STMicroelectronics Nano-2017 projects. In 2018, he joined National ASIC System Engineering Center, Southeast University,

Nanjing, China, where he is currently an Associate Professor. He is also the Guest Researcher (Maître de conférences invité) in TELECOM Paris, Institut Polytechnique de Paris, France. He has authored or coauthored two book chapters and more than 120 scientific papers, including Nature Electronics, ISSCC, JSSC, TCAS-I and TED. He has been a technical committees in IEEE-CAS society, conference TPC Member in DAC, GLSVLSI, Nanoarch, and ESREF.